# 22UIT501 - WEB PROGRAMMING ASSIGNMENT – II

**1. Design and develop a full-stack e-commerce web application named NEXUS GEAR using HTML5, CSS3, JavaScript, React.js, Java Spring Boot, and PostgreSQL. The system should include official product listings, real-time product search, user account management, wish list management, an interactive shopping cart, a secure checkout and order processing system, and a modern, responsive user interface for desktop and mobile devices.**

## CODE:

### APP.JSX:

```
import React, { useState, useEffect, useMemo } from 'react';
import Navbar from './components/Navbar';
import HeroSection from './components/HeroSection';
import CategoryBar from './components/CategoryBar';
import ProductCard from './components/ProductCard';
import CartDrawer from './components/CartDrawer';
import AuthModal from './components/AuthModal';
import CheckoutModal from './components/CheckoutModal';
import Footer from './components/Footer';

export default function App() {
  const [products, setProducts] = useState([]);
  const [categories, setCategories] = useState([]);
  const [selectedCategory, setSelectedCategory] = useState('All');
  const [searchTerm, setSearchTerm] = useState('');
  const [sortBy, setSortBy] = useState('popular');
  const [isCartOpen, setIsCartOpen] = useState(false);
  const [isAuthOpen, setIsAuthOpen] = useState(false);
  const [isCheckoutOpen, setIsCheckoutOpen] = useState(false);

  const [currentUser, setCurrentUser] = useState(() => {
    try {
      const u = localStorage.getItem('nexus_user');
      return u ? JSON.parse(u) : null;
    } catch { return null; }
  });

  const [wishlist, setWishlist] = useState([]);
  const [cartItems, setCartItems] = useState([]);
  const [promoCode, setPromoCode] = useState('');

  useEffect(() => {
    fetch('/api/products/categories')
      .then(res => res.json())
      .then(setCategories)
      .catch(() => setCategories(['All', 'Audio', 'Keyboards', 'Wearables']));
  }, []);

  useEffect(() => {
    fetchProducts();
  }, [selectedCategory, searchTerm, sortBy]);

  const handleToggleWishlist = (product) => {
    setWishlist(prev =>
```

```
    prev.some(item => item.id === product.id)
      ? prev.filter(item => item.id !== product.id)
      : [...prev, product]
  );
};

return (
  <div className="store-container">
    <Navbar
      cartCount={cartItems.reduce((acc, i) => acc + i.quantity, 0)}
      wishlistCount={wishlist.length}
      currentUser={currentUser}
      onOpenCart={() => setIsCartOpen(true)}
      onOpenAuth={() => setIsAuthOpen(true)}
      onLogout={() => setCurrentUser(null)}
      searchTerm={searchTerm}
      setSearchTerm={setSearchTerm}
    />
    <HeroSection />
    <CategoryBar
      categories={categories}
      selectedCategory={selectedCategory}
      onSelectCategory={setSelectedCategory}
    />
    <main className="catalog-section">
      <div className="catalog-header">
        <h2>All Gear & Gadgets ({products.length})</h2>
        <select value={sortBy} onChange={e => setSortBy(e.target.value)}
          className="sort-select">
          <option value="popular">Popular</option>
          <option value="priceLow">Price: Low to High</option>
          <option value="priceHigh">Price: High to Low</option>
        </select>
      </div>
      <div className="product-grid">
        {products.map(product => (
          <ProductCard
            key={product.id}
            product={product}
            onAddToCart={handleAddToCart}
            isWishlisted={wishlist.some(w => w.id === product.id)}
            onToggleWishlist={handleToggleWishlist}
```

```
            />
          ))}
        </div>
      </main>
      <CartDrawer
        isOpen={isCartOpen}
        onClose={() => setIsCartOpen(false)}
        cartItems={cartItems}
        onUpdateQuantity={handleUpdateQuantity}
        onProceedCheckout={() => { setIsCartOpen(false); setIsCheckoutOpen(true); }}
        promoCode={promoCode}
        setPromoCode={setPromoCode}
        discountAmount={discountAmount}
      />
      <CheckoutModal
        isOpen={isCheckoutOpen}
        onClose={() => setIsCheckoutOpen(false)}
        cartItems={cartItems}
        subtotal={subtotal}
        discountAmount={discountAmount}
        onOrderSuccess={() => { setCartItems([]); setPromoCode(''); }}
      />
      <AuthModal
        isOpen={isAuthOpen}
        onClose={() => setIsAuthOpen(false)}
        onAuthSuccess={setCurrentUser}
      />
      <Footer />
    </div>
  );
}
export default App;

APP.CSS:
:root {
  --primary: #3b82f6;
  --bg-dark: #0b0f19;
  --bg-card: #172033;
  --text-main: #f8fafc;
  --text-muted: #94a3b8;
  --border-color: #232f48;
  --font-family: 'Plus Jakarta Sans', Arial, sans-serif;
```

```
}
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}
body {
  font-family: var(--font-family);
  background-color: var(--bg-dark);
  color: var(--text-main);
}
.catalog-section {
  max-width: 1300px;
  margin: 0 auto;
  padding: 30px 20px;
}
.catalog-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  margin-bottom: 25px;
}
.product-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(280px, 1fr));
  gap: 25px;
}
.badge-tag {
  position: absolute;
  top: 10px;
  left: 10px;
  background: var(--primary);
  color: #fff;
  font-size: 0.75rem;
  padding: 4px 8px;
  border-radius: 4px;
}
.card-wishlist-btn {
  position: absolute;
  top: 10px;
```

```
  right: 10px;
  background: rgba(0, 0, 0, 0.6);
  border: none;
  border-radius: 50%;
  width: 32px;
  height: 32px;
  cursor: pointer;
}
.product-body {
  padding: 16px;
  display: flex;
  flex-direction: column;
  flex-grow: 1;
}
.original-price {
  font-size: 0.9rem;
  color: var(--text-muted);
  text-decoration: line-through;
}
.add-btn {
  width: 100%;
  background: var(--primary);
  color: #fff;
  border: none;
  padding: 10px;
  border-radius: 8px;
  font-weight: 600;
  cursor: pointer;
}
.cart-drawer {
  position: fixed;
  right: 0;
  top: 0;
  width: 420px;
```

AUTH.JSX:

```
import React, { useState } from 'react';
import { X } from 'lucide-react';
import { loginUser, registerUser } from '../api';
export default function AuthModal({ isOpen, onClose, onAuthSuccess }) {
```

```
  const [mode, setMode] = useState('login');
  const [formData, setFormData] = useState({ name: '', email: '', password: '' });
  const [error, setError] = useState('');
  if (!isOpen) return null;
  const handleSubmit = async (e) => {
    e.preventDefault();
    setError('');
    try {
      if (mode === 'login') {
        const data = await loginUser(formData.email, formData.password);
        onAuthSuccess(data);
        onClose();
      } else {
        const data = await registerUser(formData.name, formData.email,
          formData.password, 'CUSTOMER');
        onAuthSuccess(data);
        onClose();
      }
    } catch (err) {
      setError(err.message || 'Authentication failed');
    }
  };
  return (
    <div className="modal-overlay" onClick={onClose}>
      <div className="modal-content" onClick={e => e.stopPropagation()}>
        <div className="modal-header">
          <button onClick={onClose}><X size={20} /></button>
        </div>
        <form onSubmit={handleSubmit} className="auth-form">
          {error && <p className="error-text">{error}</p>}
          {mode === 'register' && (
            <input type="text" placeholder="Full Name" value={formData.name} onChange={e
=> setFormData({ ...formData, name: e.target.value })} required />
          )}
          <input type="email" placeholder="Email Address" value={formData.email}
onChange={e => setFormData({ ...formData, email: e.target.value })} required />
          <input type="password" placeholder="Password" value={formData.password}
onChange={e => setFormData({ ...formData, password: e.target.value })} required />
          <button type="submit" className="submit-btn">{mode === 'login' ? 'Sign In' :
'Create Account'}</button>
```

```
        </form>
      </div>
    </div>
  );
}

PRODUCT.JSX:
import React, { useState } from 'react';
import { Star, ShoppingBag, Heart } from 'lucide-react';
export default function ProductCard({ product, onAddToCart, isWishlisted,
onToggleWishlist }) {
  const specs = product.availableColors ? product.availableColors.split(',') :
['Default'];
  const [selectedSpec, setSelectedSpec] = useState(specs[0]);
  return (
    <div className="product-card">
      <div className="product-thumb-container">
        {product.badge && <span className="badge-tag">{product.badge}</span>}
        <button
          type="button"
          className="card-wishlist-btn"
          onClick={() => onToggleWishlist(product)}
        >
          <Heart size={16} color={isWishlisted ? '#ff4d4d' : '#fff'} fill={isWishlisted
? '#ff4d4d' : 'none'} />
        </button>
        <img src={product.imageUrl} alt={product.name} className="product-thumb" />
      </div>
      <div className="product-body">
        <h3 className="product-title">{product.name}</h3>
        <div className="product-rating">
          <Star size={14} fill="#f59e0b" color="#f59e0b" />
          <span>{product.rating}</span>
        </div>
        <div className="price-row">
          <span className="current-price">${product.price.toFixed(2)}</span>
          {product.originalPrice && <span className="original-price">${product.originalPrice.toFixed(2)}
</span>}
        </div>
        <div className="size-selector">
```

```
          {specs.map(s => (
            <button key={s} className={selectedSpec === s ? 'active' : ''} onClick={()
=> setSelectedSpec(s)}>{s}</button>
          ))}
        </div>
        <button className="add-btn" onClick={() => onAddToCart({ ...product,
selectedSpec, quantity: 1 })}>
          <ShoppingBag size={16} /> Add to Cart
        </button>
      </div>
    </div>
  );
}

CART.JSX:
import React, { useState } from 'react';
import { X, Trash2, ArrowRight } from 'lucide-react';
export default function CartDrawer({
  isOpen, onClose, cartItems, onUpdateQuantity, onProceedCheckout,
  promoCode, setPromoCode, discountAmount
}) {
  const [promoInput, setPromoInput] = useState(promoCode || '');
  if (!isOpen) return null;
  const subtotal = cartItems.reduce((sum, item) => sum + (item.price * item.quantity),
0);
  const shipping = subtotal >= 50.0 || subtotal === 0 ? 0.0 : 9.99;
  const finalTotal = Math.max(0, subtotal - discountAmount + shipping);
  const applyPromo = (e) => {
    e.preventDefault();
    if (promoInput.toUpperCase() === 'NEXUS20' || promoInput.toUpperCase() ===
'TECH20') {
      setPromoCode(promoInput.toUpperCase());
    }
  };
  return (
    <div className="overlay" onClick={onClose}>
      <div className="cart-drawer" onClick={e => e.stopPropagation()}>
        <div className="drawer-header">
          <h3>Shopping Cart ({cartItems.reduce((a, b) => a + b.quantity, 0)})</h3>
          <button onClick={onClose}><X size={20} /></button>
```

```
        </div>
        <div className="cart-items-list">
          {cartItems.map((item, idx) => (
            <div className="cart-item" key={idx}>
              <img src={item.imageUrl} alt={item.name} width="50" />
              <div>
                <h4>{item.name}</h4>
                <p>${item.price.toFixed(2)} x {item.quantity}</p>
                <div className="qty-control">
                  <button onClick={() => onUpdateQuantity(item.id,
item.quantity - 1)}>-</button>
                  <span>{item.quantity}</span>
                  <button onClick={() => onUpdateQuantity(item.id,
item.quantity + 1)}>+</button>
                </div>
              </div>
            </div>
          ))}
        </div>
        <form onSubmit={applyPromo} className="promo-form">
          <input type="text" placeholder="Promo code" value={promoInput} onChange={e =>
setPromoInput(e.target.value)} />
          <button type="submit">Apply</button>
        </form>
        <div className="summary">
          <p>Subtotal: ${subtotal.toFixed(2)}</p>
          {discountAmount > 0 && <p>Discount: -${discountAmount.toFixed(2)}</p>}
          <p>Shipping: {shipping === 0 ? 'FREE' : '$' + shipping.toFixed(2)}</p>
          <h4>Total: ${finalTotal.toFixed(2)}</h4>
        </div>
        <button className="checkout-btn" onClick={onProceedCheckout}>Proceed to Checkout
<ArrowRight size={16} /></button>
      </div>
    </div>
  );
}

PRODUCTCONTROLLER.JAVA:
package com.nexusgear.controller;
import com.nexusgear.model.Product;
```

```
import com.nexusgear.service.ProductService;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;
@RestController
@RequestMapping("/api/products")
@CrossOrigin(origins = "*")
public class ProductController {
  private final ProductService productService;
  public ProductController(ProductService productService) {
    this.productService = productService;
  }
  @GetMapping
  public ResponseEntity<List<Product>> getProducts(
    @RequestParam(required = false) String category,
    @RequestParam(required = false) String brand,
    @RequestParam(required = false) String search,
    @RequestParam(required = false) Double minPrice,
    @RequestParam(required = false) Double maxPrice,
    @RequestParam(required = false, defaultValue = "popular") String sortBy
  ) {
    return ResponseEntity.ok(productService.getProducts(category, brand, search,
minPrice, maxPrice, sortBy));
  }
  @GetMapping("/{id}")
  public ResponseEntity<Product> getProductById(@PathVariable Long id) {
    return
productService.getProductById(id).map(ResponseEntity::ok).orElse(ResponseEntity.notFound
().build());
  }
  @GetMapping("/categories")
  public ResponseEntity<List<String>> getCategories() {
    return ResponseEntity.ok(productService.getAllCategories());
  }
  @PostMapping
  public ResponseEntity<Product> createProduct(@RequestBody Product product) {
    return
ResponseEntity.status(org.springframework.http.HttpStatus.CREATED).body(productService.c
reateProduct(product));
  }
```

```
  @PutMapping("/{id}")
  public ResponseEntity<Product> updateProduct(@PathVariable Long id, @RequestBody
Product product) {
    return ResponseEntity.ok(productService.updateProduct(id, product));
  }
  @DeleteMapping("/{id}")
  public ResponseEntity<?> deleteProduct(@PathVariable Long id) {
    productService.deleteProduct(id);
    return ResponseEntity.ok(java.util.Map.of("message", "Product deleted"));
  }
}
```

AUTHCONTROLLER.JAVA:

```
package com.nexusgear.controller;
import com.nexusgear.dto.AuthRequest;
import com.nexusgear.dto.AuthResponse;
import com.nexusgear.dto.RegisterRequest;
import com.nexusgear.service.AuthService;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
@RestController
@RequestMapping("/api/auth")
@CrossOrigin(origins = "*")
public class AuthController {
  private final AuthService authService;
  public AuthController(AuthService authService) {
    this.authService = authService;
  }
  @PostMapping("/register")
  public ResponseEntity<AuthResponse> register(@Valid @RequestBody RegisterRequest
request) {
    return
ResponseEntity.status(HttpStatus.CREATED).body(authService.register(request));
  }
  @PostMapping("/login")
  public ResponseEntity<AuthResponse> login(@Valid @RequestBody AuthRequest request) {
    return ResponseEntity.ok(authService.login(request));
  }
```

```
  @GetMapping("/me")
  public ResponseEntity<?> getCurrentUser(@RequestParam String email) {
    return authService.getUserByEmail(email)
      .map(u -> ResponseEntity.ok(java.util.Map.of("name", u.getName(),
"email", u.getEmail(), "role", u.getRole())))
      .orElse(ResponseEntity.notFound().build());
  }
}
```

ORDERCONTROLLER.JAVA:

```
package com.nexusgear.controller;
import com.nexusgear.dto.CheckoutRequest;
import com.nexusgear.dto.OrderResponse;
import com.nexusgear.service.OrderService;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;
@RestController
@RequestMapping("/api/orders")
@CrossOrigin(origins = "*")
public class OrderController {
  private final OrderService orderService;
  public OrderController(OrderService orderService) {
    this.orderService = orderService;
  }
  @PostMapping
  public ResponseEntity<OrderResponse> checkout(@Valid @RequestBody CheckoutRequest
request) {
    return
ResponseEntity.status(HttpStatus.CREATED).body(orderService.createOrder(request));
  }
  @GetMapping("/{orderNumber}")
  public ResponseEntity<OrderResponse> getOrder(@PathVariable String orderNumber) {
    return
orderService.getOrderByNumber(orderNumber).map(ResponseEntity::ok).orElse(ResponseEntity
.notFound().build());
  }
  @GetMapping("/user/{email}")
```

```
  public ResponseEntity<List<OrderResponse>> getUserOrders(@PathVariable String email)
{
    return ResponseEntity.ok(orderService.getOrdersByCustomerEmail(email));
  }
}
```

PRODUCT.JAVA:

```
package com.nexusgear.model;
import jakarta.persistence.*;
@Entity
@Table(name = "products")
public class Product {
  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long id;
  @Column(nullable = false)
  private String name;
  @Column(nullable = false)
  private String brand;
  @Column(nullable = false)
  private String category;
  @Column(nullable = false)
  private Double price;
  private Double originalPrice;
  private String imageUrl;
  private Integer stockQuantity;
  private Double rating;
  private String badge;
  private String availableColors;
  public Product() {}
  public Long getId() { return id; }
  public void setRating(Double rating) { this.rating = rating; }
  public String getBadge() { return badge; }
  public void setBadge(String badge) { this.badge = badge; }
  public String getAvailableColors() { return availableColors; }
}
```

USER.JAVA:

```
package com.nexusgear.model;
import jakarta.persistence.*;
```

```
import java.time.LocalDateTime;
@Entity
@Table(name = "users")
public class User {
  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long id;
  @Column(nullable = false)
  private String name;
  @Column(nullable = false, unique = true)
  private String email;
  @Column(nullable = false)
  private String password;
  private String role = "CUSTOMER";
  private LocalDateTime createdAt = LocalDateTime.now();
  public User() {}
  public Long getId() { return id; }
  public void setId(Long id) { this.id = id; }
  public String getName() { return name; }
}
```

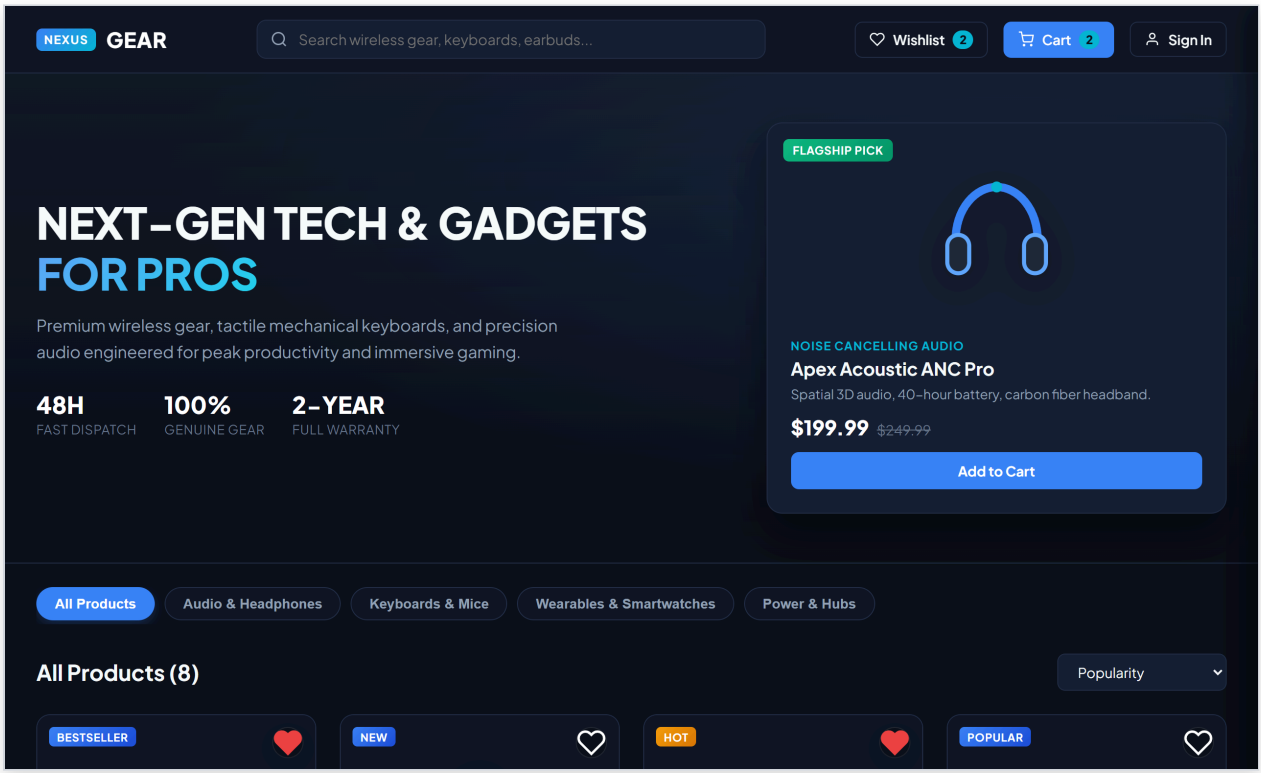
ORDER.JAVA:

```
package com.nexusgear.model;
import com.fasterxml.jackson.annotation.JsonManagedReference;
import jakarta.persistence.*;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;
@Entity
@Table(name = "orders")
public class Order {
  @Id
  @GeneratedValue(strategy = GenerationType.IDENTITY)
  private Long id;
  @Column(nullable = false, unique = true)
  private String orderNumber;
  @Column(nullable = false)
  private String customerName;
  @Column(nullable = false)
  private String customerEmail;
```

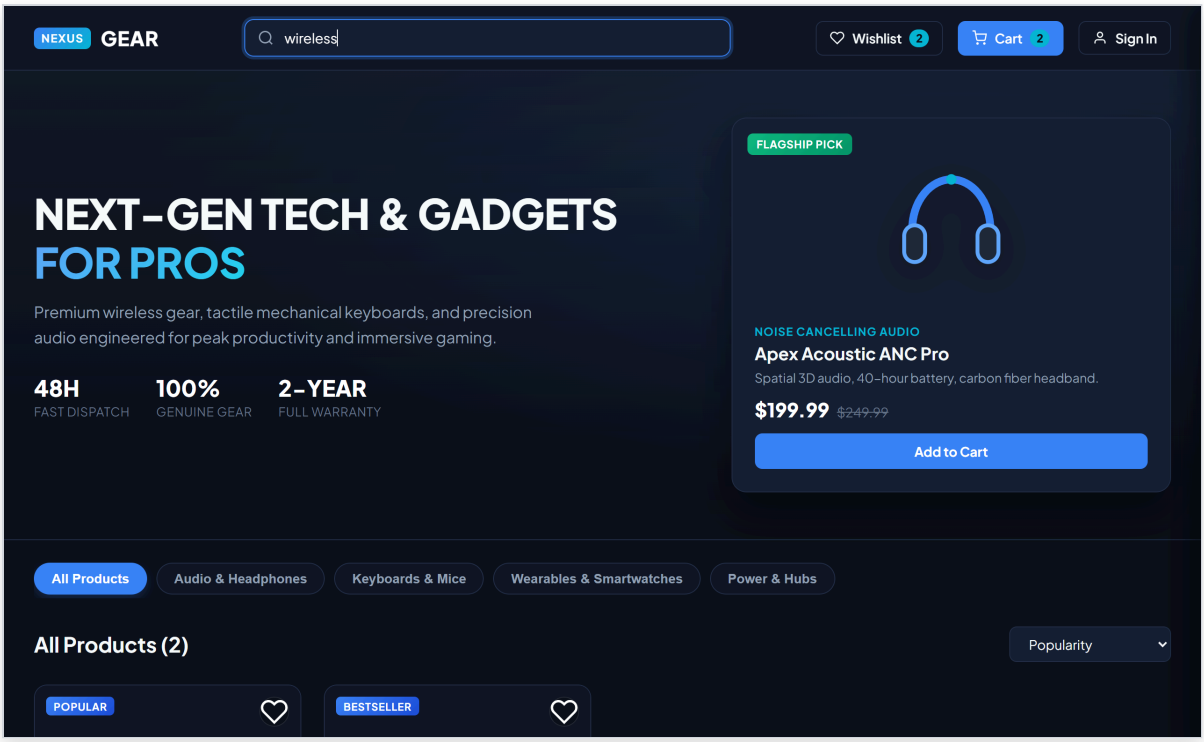
```
  @Column(nullable = false)
  private String shippingAddress;
  @Column(nullable = false)
  private Double totalAmount;
  @JsonManagedReference
  private List<OrderItem> items = new ArrayList<>();
  public Order() {}
  public Long getId() { return id; }
  public void setId(Long id) { this.id = id; }
  public String getOrderNumber() { return orderNumber; }
  public void setOrderNumber(String orderNumber) { this.orderNumber = orderNumber; }
  public String getCustomerEmail() { return customerEmail; }
  public Double getTotalAmount() { return totalAmount; }
  public void setTotalAmount(Double totalAmount) { this.totalAmount = totalAmount; }
  public String getStatus() { return status; }
  public void setStatus(String status) { this.status = status; }
  public List<OrderItem> getItems() { return items; }
  public void setItems(List<OrderItem> items) { this.items = items; }
}
```

WEBSITE IMAGES :

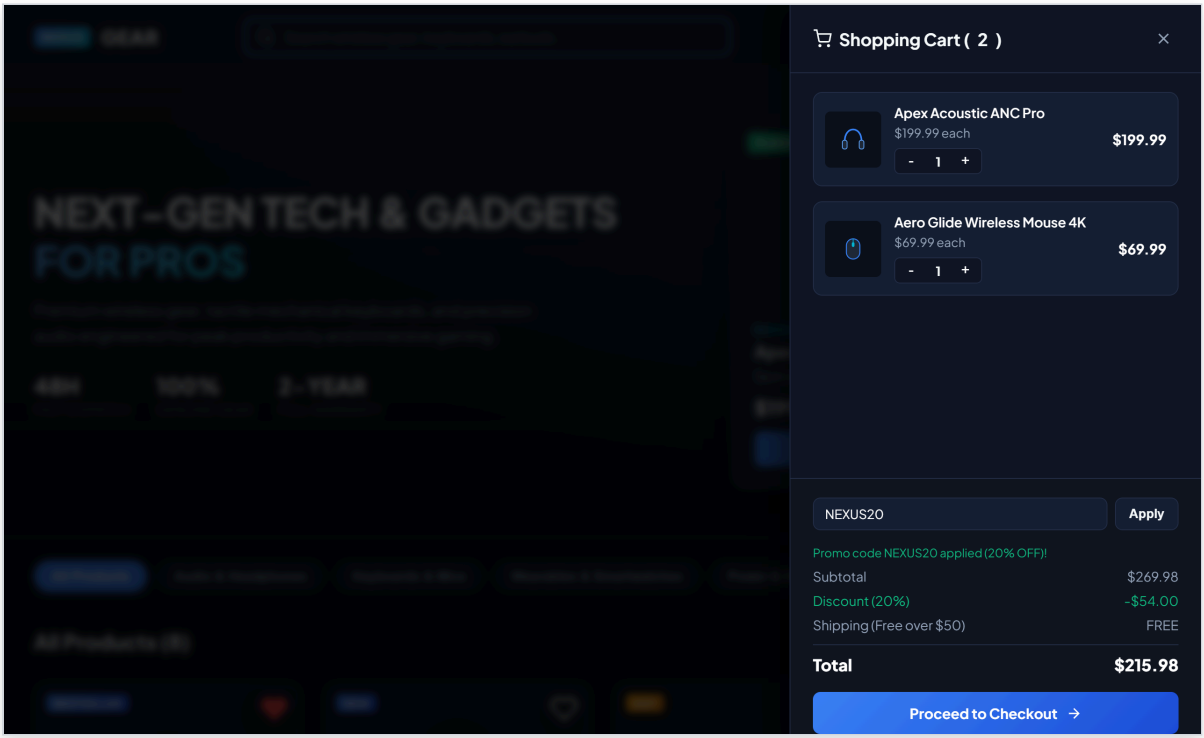
1. Home Page:



## 2. Product Search:




## 3. Shopping Cart:



## 4. Order Confirmation:

